1. Introduction

1.1 Purpose of the Project Plan

The purpose of this Project Plan is to provide a comprehensive framework for the development of a Small Business Inventory and Sales Management Tool. This document serves as a roadmap for the development team and provides stakeholders with a clear understanding of the system's technical approach, resource allocation, and risk mitigation strategies.

The plan defines the relationship between business logic and the persistence layer early in the design process. This ensures that the system is not merely a collection of interfaces, but a robust solution that maintains high data integrity and delivers actionable business insights.

1.2 Project Overview and Objectives

The Small Business Inventory & Sales Management Tool is a lightweight system created to help a small retail business manage inventory, process sales, and view basic sales reports. Many small businesses still rely on manual tracking or simple spreadsheets, which can lead to errors and difficulty in monitoring stock levels or sales performance.

This system will provide a more organized and reliable solution. It will allow users to:

- Add, update, delete, and view product information
- Record sales transactions
- Automatically update inventory quantities after each sale
- Receive low-stock alerts
- Generate daily and weekly sales summaries

The main objective of this project is to design a clean, well structured CRUD based application that demonstrates strong domain modeling and proper separation between business logic and the database layer. We also aim to ensure validation, error handling, and data integrity throughout the system.

1.3 Overview of the Document

This document is structured to guide the project from its initial conceptual phase through to technical execution and final delivery. This document is structured to guide the project from initial concept to final implementation.Sections 4-5 describe the system architecture and database design, including how data is structured and managed to ensure accuracy.Sections 6-9 outline project logistics, including the development timeline, risk assessment, and team responsibilities.Sections 10-16 focus on testing and quality assurance, detailing validation processes, testing strategies, and data handling procedures to ensure system reliability.The document concludes with the final project schedule and a glossary of key terms.

2. Project Scope

2.1 Statement of Scope

The purpose of this project is to develop a practical and dependable tool that enables small retail businesses to manage inventory and record sales accurately. The focus is on ensuring that stock levels remain consistent with daily sales activity. The tool is designed to support internal shop operations only and does not include online storefronts, payment processing, or multi-location management. It

will not include features such as online storefronts, payroll processing, or multi location management. The emphasis is on simplicity, reliability, and accurate record keeping.

2.2 System Components

2.2.1 Inventory Tracking

The inventory tracking component allows users to add, update, and manage products within the system. It provides a clear and real time view of stock levels so the business owner can immediately see what items are available. This component ensures that product details such as price and quantity are organized and easily accessible.

2.2.2 Sales Transactions

The sales transaction component supports daily checkout activities. It allows the user to select products, calculate totals, and record completed sales. Once a sale is confirmed, the system automatically updates the inventory quantity to maintain accurate stock records. This ensures that the sales data and inventory data remain consistent at all times.

2.2.3 Low Stock Alerts

The low stock alert feature monitors product quantities and compares them to predefined minimum levels. When stock falls below the set threshold, the system provides a clear notification. This helps the business owner reorder products before they run out, reducing the risk of lost sales.

2.2.4 Summary Reports and Analytics

The reporting component generates simple daily and weekly summaries based on recorded sales. It provides insights into total revenue and identifies frequently sold products. These summaries help the business owner understand performance trends and make informed restocking decisions.

2.3 Assumptions and Constraints

Assumptions:

- The system will be accessed through a modern web browser
- A stable internet connection is available
- The business owner will enter initial product data during setup

Constraints:

- The system supports only a single-store environment
- Multi-store operations are not supported
- Complex tax calculations are not included in this version

3. Process Model and Development Approach

3.1 Selected Process Model

For this project, we have chosen a hybrid development approach that combines prototyping with an increment model. We will begin by creating a detailed visual mockup of the tool to define the layout, navigation, and main workflows before moving into full development. This allows us to clearly understand how the inventory and sales screens should function from a user's perspective before building the underlying logic.

Once the prototype is reviewed and approved, development will proceed in short development cycles. Each cycle will focus on delivering a working version of a specific feature. We will start with core inventory management, then move to sales transactions, followed by low stock alerts and reporting features. At the end of each cycle, we will have a usable version of the tool that builds progressively toward the final system.

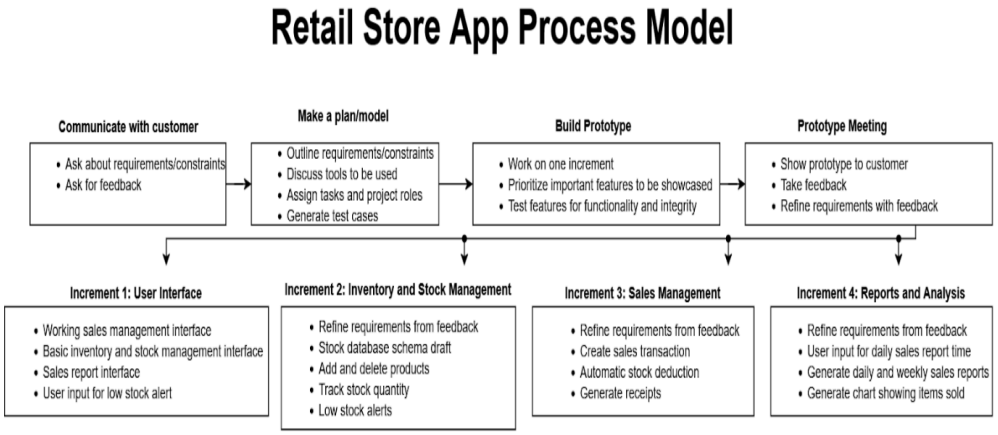

Figure: Process Model Diagram

The diagram illustrates the development process used for the retail store application, starting with communication with the customer to gather requirements and feedback, followed by planning where requirements are defined, tasks are assigned, and test cases are prepared. The process then moves into prototype development, where key features are built and tested, and a prototype review meeting is conducted to refine requirements based on feedback. The system is then developed in multiple increments, beginning with the user interface, followed by inventory and stock management, then sales management, and finally reporting and analysis features. This incremental approach ensures that the system is developed step by step, allowing continuous improvement and alignment with user needs.

3.2 Rationale for Model Selection

This model is well suited for a retail management tool because usability is just as important as technical accuracy. In a busy shop environment, the checkout process must be simple and efficient. By starting with a prototype, the business owner is given the opportunity to review and provide feedback on the layout and workflow early in the process. This reduces the risk of developing a system that is technically correct but difficult to use in practice.

The incremental structure also supports strong data integrity from the beginning. The system can first ensure that inventory tracking works correctly and that stock levels update accurately before introducing more advanced features. Building the system in stages allows each layer to be tested carefully and issues to be identified early, rather than discovered after the entire system is completed. This approach reduces overall project risk and provides flexibility to refine features, especially reporting and analytics, based on real usage and feedback.

4. System Design and Architecture Overview

The system follows a layered architecture consisting of a user interface layer, a business logic layer, and a persistence (database) layer. Each layer has a distinct responsibility to ensure separation of concerns and maintainability.

- The user interface layer handles user interactions
- The business logic layer processes rules and validations
- The persistence layer manages data storage and retrieval

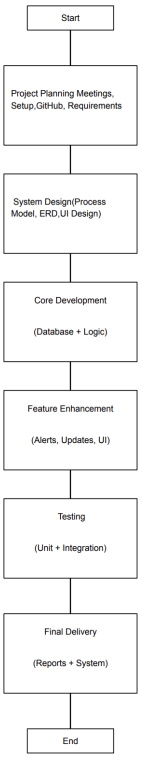

Figure : Task Network Diagram

The diagram illustrates the sequence of tasks involved in the development of the system, starting with project planning activities such as meetings, setup, and requirements gathering. It then moves into system design, including process modeling, database design, and user interface planning. After

design, core development is performed to implement the database and business logic, followed by feature enhancement where additional functionalities such as alerts and interface improvements are added. The process continues with testing, including unit and integration testing, to ensure system reliability. Finally, the project concludes with system delivery, including reporting features and the complete working system. This structured flow ensures that tasks are completed in a logical order, reducing delays and improving overall project efficiency.

4.1 Domain Modeling

At the core of this tool is a clearly defined domain model that reflects how a real retail shop operates. The system is built around three main entities: Products, Transactions, and Alerts. Products represent the physical items sold in the shop and store essential information such as name, price, and quantity. Transactions represent completed sales and record how products move out of inventory. Alerts are triggered when stock levels fall below a defined threshold.

By defining these entities and their relationships carefully, we ensure that every action in the tool mirrors real business activity. A sale is not just a number being stored; it represents an actual item leaving the shelf. This structure helps keep the digital system aligned with real world operations and reduces the chance of inconsistencies between records and physical stock.

4.2 CRUD Based Application

The tool is designed around the standard Create, Read, Update, and Delete model. This approach gives users structured and predictable control over their data. For example, when managing inventory, a user can create a new product entry, view existing stock levels, update prices or quantities, and remove or archive discontinued items.

Using this model keeps the interface consistent and easy to understand. It also makes the system easier to maintain and expand in the future. Since most business tools rely on similar patterns, this approach ensures reliability and familiarity for users.

4.3 Business Logic and Persistence Layer Integration

The architecture separates the user interface from the database through a business logic layer. This layer acts as a control point where all rules are enforced before any data is stored or modified. Users do not interact with the database directly. Instead, every action passes through this logic layer.

For example, when a sale is recorded, the system first calculates the total and verifies that enough stock is available. Only after these checks are completed does the system update the inventory and save the transaction. This separation ensures that the tool enforces business rules consistently and prevents invalid operations, such as selling an item that is no longer in stock.

4.4 Data Validation, Error Handling, and Data Integrity

The system ensures data accuracy through validation checks, preventing invalid inputs such as missing fields or negative values. Error handling mechanisms manage unexpected issues and prevent system failures. These measures maintain data integrity by ensuring that all records remain consistent, complete, and reliable. Additionally, the system prevents partial updates by completing transactions only after all validation checks are successfully passed.

5. Database Design

This section describes how data is structured, stored, and managed within the system to support efficient and reliable operations. It outlines the organization of key entities, their relationships, and how data flows between different parts of the system. The design ensures consistency, reduces redundancy, and supports accurate storage and retrieval of information.

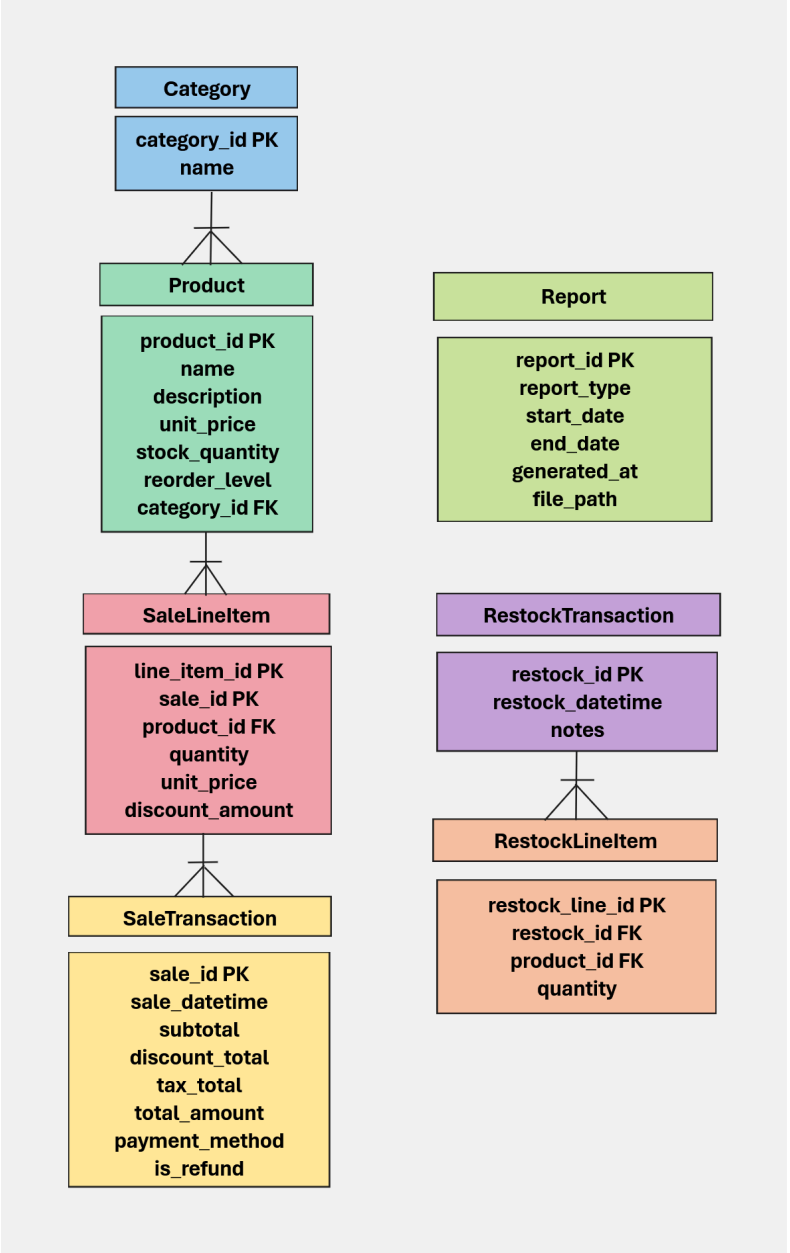

Figure : Retailsoft Entity Relationship Diagram

The diagram shows how data is structured and connected in the system. Categories organize products, and each product is linked to its category. Sales are recorded using SaleTransaction and SaleLineItem, which track items, quantities, and pricing. Inventory updates are handled through RestockTransaction and RestockLineItem to manage stock levels. The Report entity stores generated reports. This structure supports accurate tracking of inventory, sales, and data consistency across the system.

5.1 Data Model

The data model is structured to clearly represent how products move through the business over time. Each product is connected to multiple sales records, creating a one-to-many relationship between products and transactions. This allows the system to maintain a complete history of sales without mixing past records with current product details. Product information such as name and price is stored separately from transaction data, which prevents duplication and protects historical accuracy. For example, if a price changes in the future, previous sales records remain unchanged.

This separation also improves data consistency and makes the system easier to maintain and scale. By organizing data into related but independent entities, the system can efficiently retrieve, update, and analyze information without affecting other records. It also supports accurate reporting, since historical sales data always reflects the exact conditions at the time of each transaction.

5.2 Schema Design

The database schema is built around three core tables: Products, Sales, and Sales Items. The Products table stores essential details such as (Stock Keeping Unit) SKU, name, price, and current stock level. The Sales table records each completed transaction, including the date, total amount, and payment method. The Sales Items table links products to specific sales and records the quantity sold for each item, forming a relationship between products and transactions.

This structure reduces data redundancy by separating product information from transaction data and ensures accurate tracking of each sale. It also improves query performance and makes it easier to update or retrieve data without affecting other parts of the system. As the amount of data grows, this design supports scalability while maintaining efficiency and consistency.

5.3 Data Consistency Controls

To ensure accuracy, the system uses database transactions to treat each sale and inventory update as a single operation. If any part of the process fails, the entire transaction is rolled back to prevent mismatched records. Additional safeguards such as unique SKU constraints and required fields for prices and quantities help prevent invalid data from entering the system.

These controls maintain data integrity by ensuring that all related updates are completed together or not at all. They also reduce the risk of duplicate entries and incorrect inventory counts, supporting consistent and reliable system performance over time.

6. Schedule and Estimates

This section outlines the project timeline and the estimated effort required for development. It describes the estimation methodology and provides a structured breakdown of task durations and overall effort based on the team's recorded weekly activities and time tracking. The schedule reflects how the project progresses from initial planning to system development and testing.

6.1 Estimation Methodology

A bottom-up estimation approach was used to develop a realistic schedule. The project was divided into smaller tasks, such as setting meeting times, creating the GitHub repository, drafting requirements

documents, designing system components, and implementing core features. Each task was estimated individually based on its complexity, dependencies, and expected development time.Additional time was included to account for revisions after the initial prototype. Since a prototype and incremental approach is used, adjustments are expected based on feedback from team discussions, customer interviews, and document reviews. This method helps prevent unrealistic deadlines, improves planning accuracy, and ensures sufficient time is allocated for testing, debugging, and maintaining data accuracy throughout development.

6.2 Effort and Duration Estimates

The total development timeline is estimated at eight weeks. The schedule is organized based on weekly progress:

Week 1: Team formation, shared availability, GitHub setup, and product idea selection

Week 2: Requirement gathering, customer interviews, and initial documentation

Week 3: Refining documents, creating process models, and team coordination

Week 4: Drafting project plan, requirements documents, and entity diagrams

Week 5 and beyond: System design, UI development, and continued UML refinement

The project is estimated to require approximately 240 person-hours, based on the team's recorded contributions. This effort is distributed across planning, database design, user interface development, and implementation of business logic.This schedule reflects actual team activity logs and ensures that sufficient time is allocated for development, testing, and refinement, resulting in a stable and reliable system.

7. Risk Management Strategy

7.1 Likely Risks

As with any system that handles live sales and inventory data, several potential risks are identified. One major concern is data inconsistency. For example, if the system experiences a crash or network interruption during a sale, there is a risk that the transaction could be recorded without properly updating inventory levels. This could result in inaccurate stock counts and unreliable data.

Another potential risk is scope expansion during development. While prototyping is useful for refining ideas, it can also lead to additional feature requests that were not part of the original plan. If not managed carefully, this could delay the project beyond the estimated timeline.

A potential risk is incorrect user input, such as entering wrong inventory values or accidentally deleting records. These actions can lead to inaccurate data and reduce the reliability of the system if proper safeguards are not implemented.

7.2 Risk Mitigation Plan

- To reduce the risk of data inconsistencies, the system uses database transactions that treat each sale and inventory update as a single operation. If any part of the process fails, the entire transaction is rolled back to prevent partial or inaccurate updates. This ensures that inventory counts and sales records remain aligned.
- To control scope, the project follows the features defined in the project scope. Any additional ideas that arise during development are documented for future versions rather than being added to the current release.
- To address user input errors, the system includes input validation, confirmation steps for major actions, and a soft delete mechanism. This allows records to be recovered if they are removed accidentally and helps maintain long-term data reliability.

7.3 Risk Assessment Table

Risk	Likelihood	Mitigation Strategy
Data inconsistency due to system failure	Medium	Use database transactions to ensure updates are atomic (all-or-nothing)
Scope expansion during development	Medium	Follow defined project scope and document extra features for future versions
Incorrect user input or accidental deletion	Medium	Implement input validation, confirmation steps, and soft delete mechanism
Team members missing meetings	Low	Team leader sends reminders and ensures communication through group channels

Figure: Risk assessment table

The table summarizes key risks in the project along with their likelihood and mitigation strategies. It includes risks such as data inconsistency, scope expansion, user input errors, and team communication issues. Each risk is assigned a likelihood level and paired with a strategy, such as using database transactions, following defined scope, implementing validation mechanisms, and maintaining team communication. This helps ensure risks are identified early and managed effectively throughout development.

8. Project Resources

8.1 Human Resources

The development of this tool is supported by a small and focused team, with each member responsible for a specific area of the project. A project lead oversees the timeline, coordinates tasks, and communicates with the business owner to ensure the tool aligns with real operational needs.

On the technical side, a backend developer is responsible for designing the database and implementing the business logic that ensures inventory and sales data remain consistent. A frontend developer focuses on the user interface, ensuring that the inventory and sales screens are clear, efficient, and easy to use in a retail setting. Maintaining a small team allows for faster communication, better coordination, and quicker adjustments throughout the prototype and incremental development process.

8.2 Hardware and Software Resources

The project requires standard development hardware and commonly used software tools. Team members use personal computers or laptops with at least 8 GB RAM and modern processors for coding, testing, and running development environments. The system is developed for deployment on a cloud-based platform to ensure reliability and accessibility.

The application is a web-based system that runs on modern web browsers such as Google Chrome and Microsoft Edge, making it platform-independent for end users.

For software, the system is developed using the following technologies:

- Programming Languages: Python (backend), JavaScript, HTML, and CSS (frontend)
- Framework: Flask (backend web framework)
- Database: MySQL (relational database management system)
- IDE: Visual Studio Code for development and debugging
- Version Control: Git and GitHub for code management and collaboration
- Task Management: Jira for tracking tasks, assigning responsibilities, and monitoring project progress and milestones

The development environment supports efficient coding, testing, and debugging, while the target system ensures accessibility through standard web browsers. This combination of tools and platforms enables reliable development and smooth deployment of the application.

9. Project Organization

9.1 Team Structure

The project is organized around a small and focused team to maintain efficient communication and clear accountability. The structure is intentionally simple, covering three main areas: project coordination, technical development, and user interface design. This streamlined setup allows the team to respond quickly to feedback during the prototyping and incremental development phases while keeping technical and usability considerations aligned.

A flat structure supports faster decision making and reduces delays that often occur in larger teams. This approach ensures that issues can be addressed immediately without unnecessary layers of approval, improving overall efficiency and project progress.

9.2 Roles and Responsibilities

Each team member has clearly defined responsibilities to avoid confusion and duplicated work. The project lead is responsible for managing the timeline, monitoring progress, and communicating with the business owner to ensure the system meets operational needs.

The backend developer handles database design, ensures data integrity, and implements the business logic that controls inventory and sales transactions. The frontend developer focuses on designing and building the user interface, making sure the system is easy to use and that inventory and sales screens are clear and intuitive.By assigning clear ownership of each part of the system, the team ensures that both the technical components and the user experience are developed with proper attention and coordination.

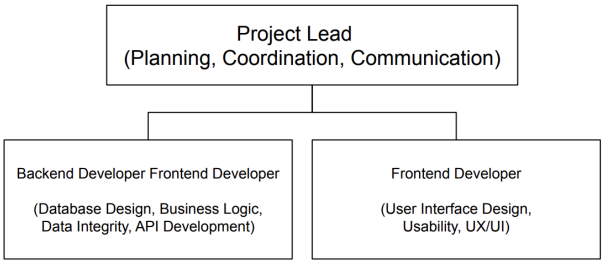

Fig: Team Structure Diagram

The diagram shows the organizational structure of the project team. The Project Lead is responsible for planning, coordination, and communication, and oversees both technical roles. The Backend developer focuses on database design, business logic, data integrity, and API development, while the Frontend Developer is responsible for user interface design and usability. This structure ensures clear responsibility and efficient collaboration within the team.

10. Testing Strategy

A comprehensive set of testing strategies will be utilized to ensure the system functions correctly, reliably, and efficiently. These strategies cover multiple levels of testing, from individual components to the complete system, to identify and resolve issues before deployment.

10.1 Unit Testing

Unit testing focuses on verifying that individual components of the system work correctly on their own. This includes testing calculations such as tax computation, total price generation, and inventory subtraction to ensure they consistently produce accurate results. By validating small pieces of logic independently, we reduce the likelihood of errors spreading through the system.

10.2 Integration Testing

Integration testing evaluates how different components of the system interact with each other. The system is tested to verify that actions performed in the user interface correctly update the database and reflect accurate inventory levels. This stage ensures that communication between the interface, business logic, and database is consistent, reliable, and free from data loss or synchronization issues.

10.3 System Testing

System testing examines the tool as a complete system in an environment that reflects real shop operations. Realistic scenarios are simulated, such as processing multiple sales transactions, handling inventory changes, and generating reports. This helps verify that the system performs reliably under normal conditions and meets overall functional requirements.

10.4 User Acceptance Testing

User Acceptance Testing is the final stage, where the shop owner or staff uses the system in a practical setting. This phase confirms that the system is easy to use, efficient, and aligned with daily business needs. It also allows users to provide feedback, ensuring that the tool not only functions correctly but also delivers real value and usability in real-world scenarios.

11. Quality Assurance and Control

- Quality assurance is integrated throughout the development process rather than treated as a final step. Automated checks are used to identify coding errors early, and regular code reviews are conducted to ensure that system logic is accurate and consistent with defined business rules. Reviewing team members' work helps reduce mistakes, improve code quality, and maintain consistency across the system.
- Quality control includes periodic verification of stored data to ensure that inventory and sales records remain accurate and up to date. System behavior is continuously monitored, and key calculations such as totals and inventory updates are validated to detect potential errors. These practices ensure that the system maintains a high level of reliability, prevents data inconsistencies, and continues to meet the operational needs of the business.

12. Change Management and Configuration Control

Change management and configuration control are used together to maintain organization, stability, and traceability throughout the development process. All source code is managed using a version control system, which tracks changes, records who made updates, and allows the system to return to a previous stable version if needed. Configuration management is treated as an ongoing activity, ensuring that all system components, including code, database structure, and documentation, remain consistent and properly maintained over time.

When new feature requests or changes arise, they are evaluated through a structured impact assessment. This review considers how the change will affect existing functionality, system stability, and the project timeline. Approved changes are documented and scheduled for future development phases rather than being implemented immediately. This approach helps control scope, prevents unexpected issues, and ensures that the system remains stable, organized, and aligned with project goals.

13. Reporting and Analytics Strategy

The purpose of the reporting and analytics component is to transform daily transaction data into meaningful insights for the business owner. Rather than presenting raw numbers, the tool summarizes information in a clear and simple format that allows the owner to quickly understand how the shop is performing. The focus is on providing practical insights that support everyday decision making.

13.1 Key Performance Indicators

The tool tracks a set of core performance indicators that reflect the overall health of the business. These include total daily revenue, average transaction value, and profit margins by product category. Monitoring these metrics allows the owner to identify profitable days, evaluate pricing effectiveness, and understand which product categories contribute most to overall revenue.

13.2 Inventory and Sales Trends

In addition to summary metrics, the system analyzes patterns in sales and inventory activity. It identifies products that sell quickly as well as items that remain in stock for extended periods. This trend analysis helps the owner make informed purchasing decisions and adjust stock levels based on actual demand. By connecting these insights with the low stock alert system, the tool supports smarter inventory planning and reduces the risk of overstocking or running out of popular items.

14. Project Schedule

The project follows a structured weekly timeline based on the prototype and incremental development approach outlined earlier. This schedule allows sufficient time for development, testing, and feedback from the business owner. By dividing the project into focused phases, the team can deliver steady progress while maintaining quality and reliability.

14.1 Milestone Timeline(assumed for draft)

The project timeline is divided into structured weekly phases based on actual team activities and recorded progress. Each phase reflects collaboration, planning, design, and development efforts carried out throughout the project.

During Weeks 1 and 2, the focus is on team coordination, planning, and initial setup. The team establishes communication, shares availability, creates the GitHub repository, and finalizes the product name. Requirements questions are discussed, and initial customer interviews are reviewed to better understand system needs. Early drafts such as process models and project outlines are also developed during this phase.

Weeks 3 and 4 focus on refining project documents and design elements. The team improves the process model, reviews the project roadmap, and finalizes key components such as the team logo and requirements documentation. Entity relationship diagrams and structured planning documents are created to prepare for development. Continuous team discussions ensure alignment and progress tracking.

Weeks 5 and 6 emphasize implementation and interface design. The team begins working on input requirements, develops the graphical user interface design, and updates navigation diagrams. Feedback is incorporated to refine the system structure, ensuring that both functionality and usability are improved.

14.2 Deliverables Schedule(assumed for draft)

At the end of each phase, specific deliverables are produced to demonstrate progress and maintain accountability.

- By the end of Week 2, the team completes initial planning tasks, including repository setup, product definition, and early process models.
- By the end of Week 4, key project documents are finalized, including refined process models, requirements documentation, and system design diagrams such as entity relationships.
- By the end of Week 6, interface designs and system structure updates are completed, including navigation diagrams and initial implementation of system components.

The final phase delivers a fully integrated system, supported by testing and validation to ensure reliability, usability, and readiness for real world application.

15. Glossary and References

15.1 Glossary(will be add more)

CRUD refers to Create, Read, Update, and Delete, which are the four fundamental operations used to manage stored data.

Domain Model describes a structured representation of the key entities within the system and how they relate to one another in a real business context.

Data Integrity refers to the accuracy and consistency of data over its entire lifecycle, ensuring that records remain reliable and free from corruption.

Persistence Layer is the part of the system responsible for saving and retrieving data from permanent storage, such as a database.

Unit Testing is a testing method where individual components of the system are verified independently to confirm they function correctly.

User Acceptance Testing is the final stage of testing in which the business owner or end user validates that the tool meets real operational needs.

Relational Database refers to a structured database that organizes data into tables with defined relationships, allowing efficient storage and retrieval of related information.

Primary Key is a unique identifier assigned to each record in a table to ensure that every entry can be distinctly identified.

Foreign Key is a field used to link one table to another, establishing relationships between different data entities.

Transaction refers to a group of database operations that are executed as a single unit, ensuring that all changes are completed successfully or none are applied.

Version Control is a system used to track changes in code over time, allowing developers to manage updates and restore previous versions if needed.

User Interface (UI) refers to the visual elements of the system that users interact with, such as screens, buttons, and forms.

Business Logic defines the rules and processes that control how data is handled within the system, including calculations, validations, and workflows.

Inventory Management refers to the process of tracking product quantities, updating stock levels, and ensuring availability of items.

Low Stock Alert is a feature that notifies users when product quantities fall below a predefined threshold, helping prevent stock shortages.

15.2 References

Software Engineering: a practitioner's Approach by Bruce Maxim and roger Pressman